ТЕХНИЧЕСКИЙ РАЗБОР КОДОВОЙ БАЗЫ • АРХИТЕКТУРА СКРИПТОВ

⚙ Логика работы скриптов проекта «AI-Агент Логистика»

Подробное описание модулей бэкенда (FastAPI, Data Loader, ML Predictor, Queue Optimizer, Hybrid AI Agent) и клиентского дашборда

Обзор модульной архитектуры

Система построена по модульному принципу и разделена на точку входа (`run.py`), серверную часть в директории `backend/` и веб-интерфейс в `frontend/` :

Файл / Скрипт	Основной класс / Функции	Назначение в цепочке обработки данных
`run.py`	`is_port_in_use()`	Проверка занятости порта `8000` , защита от повторного запуска и старт сервера `uvicorn` .
`backend/main.py`	`FastAPI app`	Инициализация всех ядер при старте, маршрутизация REST API и отдача статики фронтенда.
`backend/data_loader.py`	`LogisticsDataStore`	Загрузка 8 CSV-таблиц, парсинг рабочих часов, расчет индекса перегруженности и сбор очередей.
`backend/ml_predictor.py`	`DetentionMLPredictor`	Обучение `HistGradientBoostingRegressor` , прогноз по 24 слотам и вероятностная аналитика Гамма-распределения.
`backend/queue_optimizer.py`	`QueueOptimizerEngine`	Скоринг приоритетов (0–99), распределение фур по активным бригадам/докам и балансировка между хабами города.
`backend/ai_agent.py`	`HybridLogisticsAgent`	Автономное аналитическое ядро + интеграция с внешними LLM (Gemini, OpenAI, Ollama) и вызов инструментов.
`frontend/app.js`	`loadOverview()` , `runOptimization()`	Запросы к API, отрисовка KPI-карточек, графиков `Chart.js` , фильтрация расписания и чат с AI-диспетчером.

1. Точка входа и API-сервер (`run.py` и `backend/main.py`)

Скрипт запуска `run.py`

- Функция `is_port_in_use(host, port)` открывает TCP-сокет к `127.0.0.1:8000` с таймаутом 1 секунда.
- Если сервер уже запущен, скрипт не падает с ошибкой занятого порта, а автоматически открывает вкладку браузера через `webbrowser.open(url)` и завершает процесс (`sys.exit(0)`).
- Если порт свободен — запускает `uvicorn.run("backend.main:app", host="127.0.0.1", port=8000)` .

Сервер `backend/main.py`

- В хуке `@app.on_event("startup")` вызывает `store.load_all()` и обучает модель `predictor.train(sample_size=45000)` .
- Эндпоинты REST API:
  - `GET /api/overview` — агрегированные KPI сети и список всех 50 складов.
  - `POST /api/optimize` — оптимизация очереди выбранного терминала.
  - `POST /api/predict` — ML-прогноз простоя и поиск лучшего слота.
  - `POST /api/agent/chat` и `/api/agent/config` — управление AI-агентом.

2. Загрузка и обогащение данных (`backend/data_loader.py`)

Класс `LogisticsDataStore` отвечает за чтение реляционного датасета и предварительный расчет сетевых метрик:

1. **Парсинг расписания складов (`parse_operating_hours`)**: переводит строковые режимы работы (`"24/7"` , `"8AM-5PM"` , `"7AM-7PM"` , `"6AM-10PM"`) в числовые границы `(open_hour, close_hour, is_24_7)` .
2. **Расчет статистик и узких мест (`load_all`)**:
   - Загружает таблицы `facilities.csv` , `loads.csv` , `trips.csv` , `drivers.csv` , `trucks.csv` , `customers.csv` , `routes.csv` и `delivery_events.csv` .
   - Вычисляет отклонение фактического прибытия от расписания (`arrival_delay_min`) и булев флаг приезда в закрытые часы склада (`out_of_hours`).
   - Группирует терминалы по городам (`city_facilities`), формируя для каждого склада список партнерских хабов в том же городе (`alternative_city_hubs`).
   - Рассчитывает **индекс перегруженности (`congestion_index` от 0 до 100)** по формуле, объединяющей средний простой, долю внеурочных прибытий и плотность рейсов на один док:

     ```
     congestion_index = min(100.0, (avg_detention / 120.0) * 45.0 + (out_of_hours_rate * 35.0) + min(20.0, density_score * 8.0))
     ```
3. **Выборка очереди терминала (`get_facility_queue_events`)**: фильтрует события за окно `window_days` (если дата не передана, выбирает самый загруженный день терминала) и обогащает каждую строку через `LEFT JOIN` данными о грузе, рейсе, клиенте, водителе и тягаче.

3. Машинное обучение и Гамма-аналитика SLA (`backend/ml_predictor.py`)

Класс `DetentionMLPredictor` объединяет регрессионную модель градиентного бустинга и вероятностное моделирование рисков:

Обучение `HistGradientBoostingRegressor` (`train`)

- Обучается на **15 признаках**: число доков, часы открытия/закрытия, флаг 24/7, коды типа склада, события, груза и контракта, вес (lbs), число паллет, час прибытия, день недели, задержка прибытия, флаг внерабочего часа и часовая нагрузка на док.
- Целевая переменная `y` комбинирует исторический простой (`65%`) и физический штраф (`35%`) за закрытые ворота, тоннаж и количество паллет.
- Метод `estimate_service_duration_minutes()` вычисляет физическое время разгрузки у дока (от 25 до 120 мин) с учетом базового норматива, веса, паллет и `+12 мин` на проверку температурного режима для рефрижераторов.

Прогноз слотов и Гамма-распределение

- **24-часовой профиль (`predict_detention_and_best_slot`)**: рассчитывает ожидаемый простой для каждого часа суток (`00:00–23:00`), добавляет время ожидания открытия ворот при приезде в закрытые часы и выбирает оптимальный слот с минимальной очередью.
- **Аналитика SLA (`compute_gamma_analytics`)**: методом моментов оценивает параметры Гамма-распределения по историческим простоям склада (`alpha = mean² / var` , `theta = var / mean`) и через `scipy.stats.gamma` вычисляет квантили **P90**, **P95** и вероятность нарушения 2-часового SLA: `P(T > 120 мин)` .

4. Алгоритм оптимизации очереди и доков (`backend/queue_optimizer.py`)

Класс `QueueOptimizerEngine` управляет перестроением очереди и назначением фур на конкретные доки:

Шаг 1: Многофакторный скоринг приоритета (`compute_priority_score` — от 15 до 99 баллов)

- **Тип груза**: `Refrigerated` (рефрижераторы) получают **+34 балла** для защиты холодовой цепи, `Dry Van` — **+10 баллов**.
- **Уровень контракта**: `Dedicated` (**+24**), `Contract` (**+16**), `Spot` (**+6**).
- **Риск срыва SLA и задержка**: до **+26 баллов** пропорционально текущему простою и опозданию рейса.
- **Ценность и масса груза**: до **+16 баллов** на основе выручки (`revenue`) и веса (`weight_lbs`).
- Категории приоритета: **CRITICAL (≥75)** **HIGH (≥55)** **MEDIUM (≥38)** **LOW (<38)** .

Шаг 2: Поволновая диспетчеризация и балансировка хабов (`optimize_facility_queue`)

- **Стресс-тест (`surge_extra_trucks`)**: при активации инжектирует дополнительные внеплановые фуры в утренние пики (`08:00–09:00`) и вечерние часы закрытия (`16:00–19:00`).
- **Операционные волны**: внутри каждого дня прибытия делятся на 3-часовые волны (`wave_id = hour // 3`) и сортируются по убыванию `priority_score` .
- **Контроль занятости бригад (`bay_free_times`)**: отслеживает момент освобождения каждого активного дока с учетом 5-минутного технологического окна между фурами.
- **Правила маршрутизации каждой фуры**:
  1. *Прибытие в закрытые часы + доступен открытый хаб в том же городе* (при `priority_score ≥ 52` или `orig_det ≥ 70`) → **`CITY_HUB_REROUTE`** (перенаправление в соседний открытый хаб с 15-минутным трансфером).
  2. *Прибытие в закрытые часы без переутинга* → **`SHIFTED_TO_OPEN_HOURS`** (перенос слота на первый доступный док в момент открытия склада).
  3. *Прибытие в рабочие часы* → назначение на самый ранний свободный док (`np.argmin(bay_free_times)`). Рейсы `CRITICAL` и `HIGH` получают статус **`PRIORITY_FAST_TRACK`** , остальные — сглаживание пика (**`SLOT_SMOOTHED`** / **`SLOT_OPTIMIZED`**).
- **Расчет ROI и экологии**: сэкономленные минуты переводятся в финансовую экономию (`$1.25/мин` = `$75/ч`), топливо на холостом ходу (`0.85 гал/ч`) и снижение выбросов CO_2 (`10.18 кг/гал`).

5. Гибридный AI-Агент (`backend/ai_agent.py`) и Фронтенд (`frontend/app.js`)

Гибридный AI-Диспетчер (`HybridLogisticsAgent`)

- **Парсинг локаций (`_detect_facility_from_text`)**: распознает в запросе код `FAC000XX` или название города на русском/английском (например, «Атланта» → `FAC00012` , «Филадельфия» → `FAC00008`).
- **Вызов инструментов (Tool Execution)**: при каждом сообщении выполняет 3 реальных инструмента: `optimize_facility_queue` , `scan_network_bottlenecks` и `predict_detention_and_best_slot` .
- **Автономный и LLM режимы**: по умолчанию формирует аналитический ответ встроенным детерминированным ядром (`_build_analytical_response`). При подключении ключей `Gemini` , `OpenAI` или локальной `Ollama` передает JSON-телеметрию в LLM с автоматическим фоллбэком при сбое сети.

Клиентский скрипт (`frontend/app.js`)

- При инициализации страницы (`DOMContentLoaded`) последовательно вызывает `loadOverview()` , `runOptimization()` и `runMLPrediction()` .
- Строит интерактивный двухслойный график `Chart.js` (сравнение среднего простоя по часам До и После AI-оптимизации).
- Управляет фильтрацией таблицы расписания доков по типам принятых решений (Все рейсы, Fast-Track, Переутинг в хабы, Перенос слотов) и отображает в чате плашки вызванных агентом инструментов.